

Hamming(7,4) Error Detection and Correction

A Complete System Design and Implementation Guide

RTL Design | Theory | Verification

GitHub Repository: [hamming74-system](#)
Language: Verilog HDL

2026

Table of Contents

1. Introduction	3
2. Theory of Hamming(7,4) Codes.....	4
2.1 Error Detection and Correction Fundamentals.....	4
2.2 The Hamming Bound	4
2.3 Parity Bit Positions	4
2.4 Parity Bit Computation.....	5
2.5 Syndrome Decoding	5
2.6 Generator and Parity-Check Matrices	6
3. Worked Example: End-to-End Trace	7
3.1 Given: Data Input	7
3.2 Step 1: Compute Parity Bits	7
3.3 Step 2: Assemble the Codeword.....	7
3.4 Step 3: Inject a Single-Bit Error	7
3.5 Step 4: Compute the Syndrome	7
3.6 Step 5: Correct the Error.....	8
3.7 Step 6: Extract Data Bits	8
3.8 Summary Trace Table	8
4. Repository Code Explanation.....	9
4.1 Repository Structure	9
4.2 Module: hamming74_encoder.....	9
4.3 Module: error_injector.....	10
4.4 Module: hamming74_decoder.....	11
4.5 Module: hamming74_system (Top-Level).....	12
4.6 Module Connectivity Summary	13
5. Planned Next Steps.....	14
6. Conclusion.....	15

1. Introduction

In any digital communication or data storage system, errors are an unavoidable reality. Noise, electromagnetic interference, manufacturing imperfections, and cosmic ray events can all flip a single bit from a 0 to a 1 or vice versa. Without a mechanism to detect and recover from such errors, even the most carefully engineered systems can produce incorrect results with potentially catastrophic consequences.

The Hamming(7,4) code, invented by Richard W. Hamming at Bell Labs in 1950, was one of the earliest systematic solutions to this problem. It remains one of the most elegant and pedagogically important error-correcting codes ever devised. The code takes 4 bits of raw data and produces a 7-bit codeword by appending 3 parity bits. Crucially, those parity bits are positioned and computed in a way that allows not only detection of a single-bit error but also its precise correction — without any retransmission.

This document provides a complete, ground-up understanding of the Hamming(7,4) system implemented in this repository. It covers:

- The mathematical theory of linear block codes and Hamming's construction
- A detailed worked example tracing data through encoding, corruption, and decoding
- A module-by-module explanation of the Verilog RTL implementation
- A description of the planned verification and automation infrastructure

Whether you are encountering error-correcting codes for the first time or reviewing an RTL implementation for a digital design project, this guide is intended to give you full confidence in the system — from first principles all the way to synthesisable hardware.

2. Theory of Hamming(7,4) Codes

2.1 Error Detection and Correction Fundamentals

A binary block code takes k information bits and maps them to an n -bit codeword. The ratio k/n is called the code rate and represents the efficiency of the encoding. In a Hamming(7,4) code, $n = 7$ and $k = 4$, giving a code rate of $4/7 \approx 0.571$. The remaining $r = n - k = 3$ bits are redundancy bits, also called parity or check bits.

The fundamental quantity that governs error correction capability is the Hamming distance between two codewords — the number of bit positions in which they differ. For a code to detect up to t errors, its minimum Hamming distance $d_{\min}$ must satisfy:

$$d_{\min} \geq t + 1$$

For a code to correct up to t errors, the requirement is stricter:

$$d_{\min} \geq 2t + 1$$

The Hamming(7,4) code has $d_{\min} = 3$, which means it can detect up to 2-bit errors and correct exactly 1-bit errors. This property is known as single-error correcting (SEC).

2.2 The Hamming Bound

How many redundancy bits r are needed to correct any single-bit error in a codeword of length n ? Each n -bit codeword can be in error in exactly n ways (one per bit position), or it can be correct. So the r bits must be able to address $n + 1$ distinct states:

$$2^r \geq n + 1 = (k + r) + 1$$

For $k = 4$ data bits: $2^r \geq 4 + r + 1$. Testing $r = 3$ gives $2^3 = 8 \geq 4 + 3 + 1 = 8$. This holds with equality, making the Hamming(7,4) code a perfect code — every possible bit pattern is either a valid codeword or is exactly one bit-flip away from a unique valid codeword.

2.3 Parity Bit Positions

In Hamming's scheme, parity bits are placed at positions that are powers of two: positions 1, 2, and 4. Data bits occupy the remaining positions: 3, 5, 6, and 7. The bit-position numbering is 1-based, from least significant to most significant.

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1
d0	d1	d2	p4	d3	p2	p1
Data	Data	Data	Parity	Data	Parity	Parity

(Blue = data bit, Orange = parity bit)

2.4 Parity Bit Computation

Each parity bit P_i covers all bit positions whose binary representation has a 1 in the same column as i . Concretely:

P1 (bit 1) covers positions 1, 3, 5, 7 — those whose bit-0 is 1.

P2 (bit 2) covers positions 2, 3, 6, 7 — those whose bit-1 is 1.

P4 (bit 4) covers positions 4, 5, 6, 7 — those whose bit-2 is 1.

Each parity bit is set so that the XOR of all the positions it covers equals zero (even parity). Using d_3, d_2, d_1, d_0 for the data bits at positions 3, 5, 6, 7 respectively:

$$P_1 = d_3 \oplus d_2 \oplus d_0$$

$$P_2 = d_3 \oplus d_1 \oplus d_0$$

$$P_4 = d_2 \oplus d_1 \oplus d_0$$

2.5 Syndrome Decoding

At the receiver, three syndrome bits S_1, S_2, S_4 are computed by re-evaluating the parity of each coverage group including the parity bit itself. If the received codeword has no error, all syndrome bits are zero. If a single-bit error occurred at position e , the syndrome equals e in binary:

$$S_1 = r_1 \oplus r_3 \oplus r_5 \oplus r_7$$

$$S_2 = r_2 \oplus r_3 \oplus r_6 \oplus r_7$$

$$S_4 = r_4 \oplus r_5 \oplus r_6 \oplus r_7$$

$$\text{Syndrome} = \{ S_4, S_2, S_1 \}$$

The syndrome value directly gives the position of the erroneous bit. If the syndrome is non-zero, the bit at position syndrome is simply flipped to correct the error. The 4 data bits can then be extracted from positions 3, 5, 6, and 7 of the corrected codeword.

2.6 Generator and Parity-Check Matrices

The Hamming(7,4) code is a linear code. Its encoding can be written compactly using a generator matrix G . Each data vector $u = [d_0, d_1, d_2, d_3]$ is encoded to codeword $c = u \cdot G$. The parity-check matrix H has the property that $H \cdot c^T = 0$ for any valid codeword and $H \cdot r^T = \text{syndrome}$ for any received word r .

The parity-check matrix for the Hamming(7,4) code (columns ordered $p_1, p_2, d_3, p_4, d_2, d_1, d_0$) is:

	b1	b2	b3	b4	b5	b6	b7
S1	1	0	1	0	1	0	1
S2	0	1	1	0	0	1	1
S4	0	0	0	1	1	1	1

Each column of H is the binary representation of the corresponding bit position. This is what makes syndrome decoding so elegant — the syndrome directly encodes the error location.

3. Worked Example: End-to-End Trace

3.1 Given: Data Input

Let the 4-bit input data be:

$$\text{data_in} = 1011 \quad (d_3=1, d_2=0, d_1=1, d_0=1)$$

3.2 Step 1: Compute Parity Bits

Apply the three parity equations:

$$P_1 = d_3 \oplus d_2 \oplus d_0 = 1 \oplus 0 \oplus 1 = 0$$

$$P_2 = d_3 \oplus d_1 \oplus d_0 = 1 \oplus 1 \oplus 1 = 1$$

$$P_4 = d_2 \oplus d_1 \oplus d_0 = 0 \oplus 1 \oplus 1 = 0$$

3.3 Step 2: Assemble the Codeword

Place bits into their correct positions (positions 1 through 7):

Pos 7	Pos 6	Pos 5	Pos 4	Pos 3	Pos 2	Pos 1
d0 = 1	d1 = 1	d2 = 0	P4 = 0	d3 = 1	P2 = 1	P1 = 0

Therefore the encoded 7-bit codeword is:

$$\text{code_out} = 1\,1\,0\,0\,1\,1\,0 \quad (\text{bits 7 down to 1}) = 7'b1100110$$

3.4 Step 3: Inject a Single-Bit Error

Suppose a single-bit error occurs at position 5 (the d2 bit). We flip bit position 5 from 0 to 1:

$$\text{corrupted} = 1\,1\,1\,0\,1\,1\,0 = 7'b1110110$$

3.5 Step 4: Compute the Syndrome

At the receiver, apply the three syndrome equations to the corrupted codeword $r = [r_7, r_6, r_5, r_4, r_3, r_2, r_1] = [1, 1, 1, 0, 1, 1, 0]$:

$$S_1 = r_1 \oplus r_3 \oplus r_5 \oplus r_7 = 0 \oplus 1 \oplus 1 \oplus 1 = 1$$

$$S_2 = r_2 \oplus r_3 \oplus r_6 \oplus r_7 = 1 \oplus 1 \oplus 1 \oplus 1 = 0$$

$$S_4 = r_4 \oplus r_5 \oplus r_6 \oplus r_7 = 0 \oplus 1 \oplus 1 \oplus 1 = 1$$

$$\text{Syndrome} = \{S_4, S_2, S_1\} = \{1, 0, 1\} = 3'b101 = 5$$

The syndrome is 5 — exactly the position of the injected error. This is the central elegance of the Hamming code: the syndrome decodes the error location directly, requiring no search.

3.6 Step 5: Correct the Error

Flip bit position 5 of the received word:

$$\text{corrected} = \text{corrupted} \oplus (1 \ll (\text{syndrome} - 1))$$

$$\text{corrected} = 1110110 \oplus 0010000 = 1100110 = 7'b1100110$$

This matches the original codeword exactly, confirming successful correction.

3.7 Step 6: Extract Data Bits

From the corrected codeword, extract the data from positions 7, 6, 5, 3 (the non-parity positions):

$$\text{data_out} = \{ \text{bit7}, \text{bit6}, \text{bit5}, \text{bit3} \} = \{ 1, 1, 0, 1 \} = 4'b1011$$

This matches the original input `data_in = 1011`, demonstrating complete end-to-end correctness.

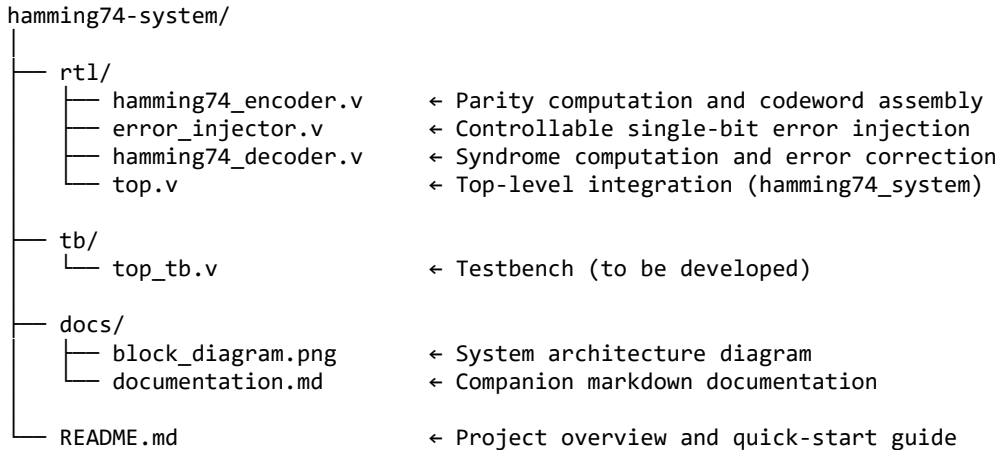
3.8 Summary Trace Table

Stage	Value	Notes
<code>data_in</code>	<code>4'b1011</code>	<code>d3=1, d2=0, d1=1, d0=1</code>
<code>code_out (encoded)</code>	<code>7'b1100110</code>	<code>P1=0, P2=1, P4=0</code> appended
<code>corrupted_data</code>	<code>7'b1110110</code>	Error at position 5
<code>syndrome</code>	<code>3'b101 = 5</code>	Points to bit position 5
<code>corrected_code</code>	<code>7'b1100110</code>	Bit 5 flipped back
<code>data_out</code>	<code>4'b1011</code>	Matches original — SUCCESS

4. Repository Code Explanation

4.1 Repository Structure

The repository is organised into four directories, each with a clearly defined purpose:



The `rtl/` directory contains the synthesisable design. Each module is placed in its own file for clarity and modular simulation. The `tb/` directory will contain the verification environment described in Section 5.

4.2 Module: `hamming74_encoder`

The encoder is a purely combinational module. Given 4 data bits, it computes the three parity bits and assembles the complete 7-bit codeword.

```

module hamming74_encoder(
    input  [3:0] data_in,
    output [6:0] code_out
);

    wire p1, p2, p4;

    assign p1 = data_in[3] ^ data_in[2] ^ data_in[0];
    assign p2 = data_in[3] ^ data_in[1] ^ data_in[0];
    assign p4 = data_in[2] ^ data_in[1] ^ data_in[0];

    assign code_out = {
        data_in[0], // position 7
        data_in[1], // position 6
        data_in[2], // position 5
        p4,         // position 4
        data_in[3], // position 3
        p2,         // position 2
        p1          // position 1
    };

endmodule

```

Key observations about this implementation:

- The data input follows the convention $\text{data_in}[3:0] = \{ d3, d2, d1, d0 \}$, where $d3$ is the most significant data bit.
- Parity bit $P1$ XORs $\text{data_in}[3]$, $\text{data_in}[2]$, and $\text{data_in}[0]$ — corresponding to $d3$, $d2$, and $d0$, which occupy positions 3, 5, and 7 (all have bit-0 = 1).
- Parity bit $P2$ XORs $\text{data_in}[3]$, $\text{data_in}[1]$, and $\text{data_in}[0]$ — corresponding to positions 3, 6, and 7 (all have bit-1 = 1).
- Parity bit $P4$ XORs $\text{data_in}[2]$, $\text{data_in}[1]$, and $\text{data_in}[0]$ — corresponding to positions 5, 6, and 7 (all have bit-2 = 1).
- The code_out concatenation places bits from index 6 (position 7) down to index 0 (position 1), so $\text{code_out}[0] = p1$ (position 1) and $\text{code_out}[6] = d0$ (position 7).
- The module is purely combinational with no registers or clocks — it can be synthesised directly as a cloud of XOR gates.

4.3 Module: error_injector

The error injector is a utility module used in simulation and testing. It allows a single bit to be flipped at a controllable position, emulating a channel error.

```
module error_injector(
    input  [6:0] code_in,
    input  [2:0] error_pos,
    input      enable,
    output [6:0] code_out
);

    reg [6:0] error_mask;

    always @(*) begin
        error_mask = 7'b0000000;
        if(enable)
            error_mask[error_pos] = 1'b1;
        end

    assign code_out = code_in ^ error_mask;

endmodule
```

Key observations:

- When enable is low, error_mask is all zeros, and code_out equals code_in — no corruption.
- When enable is high, a single 1 is placed at bit error_pos of the mask. XORing the codeword with a mask that has exactly one 1 flips exactly one bit — the definition of a single-bit error.

- The error_pos input is 3 bits wide (range 0 to 7). In practice, meaningful error positions are 0 to 6, corresponding to the 7 bit positions of the codeword.
- This module exists only for verification purposes and would not be instantiated in a production design.

4.4 Module: hamming74_decoder

The decoder is the most complex of the three leaf modules. It computes the syndrome, detects errors, corrects a single-bit error, and extracts the 4 original data bits.

```
module hamming74_decoder(
    input  [6:0] code_in,
    output [3:0] data_out,
    output [2:0] syndrome,
    output          error_detected
);

    reg [6:0] corrected_code;

    wire s1, s2, s4;

    // Syndrome calculation
    assign s1 = code_in[0] ^ code_in[2] ^ code_in[4] ^ code_in[6];
    assign s2 = code_in[1] ^ code_in[2] ^ code_in[5] ^ code_in[6];
    assign s4 = code_in[3] ^ code_in[4] ^ code_in[5] ^ code_in[6];

    assign syndrome = {s4, s2, s1};
    assign error_detected = |syndrome;

    // Error correction
    always @(*) begin
        corrected_code = code_in;
        if(syndrome != 3'b000)
            corrected_code[syndrome - 1] = ~corrected_code[syndrome - 1];
    end

    // Extract original data
    assign data_out = {
        corrected_code[2], // D3 (position 3 → index 2)
        corrected_code[4], // D2 (position 5 → index 4)
        corrected_code[5], // D1 (position 6 → index 5)
        corrected_code[6]  // D0 (position 7 → index 6)
    };

endmodule
```

A careful analysis of each portion of the decoder:

Syndrome Wires. The signals `s1`, `s2`, and `s4` correspond to `S1`, `S2`, and `S4` from the theory. Each is the XOR of the bits at all positions covered by the respective parity bit. Because `code_in` is indexed 0 to 6, position `p` maps to `code_in[p-1]`:

- $s1 = \text{code_in}[0] \oplus \text{code_in}[2] \oplus \text{code_in}[4] \oplus \text{code_in}[6]$ — positions 1, 3, 5, 7
- $s2 = \text{code_in}[1] \oplus \text{code_in}[2] \oplus \text{code_in}[5] \oplus \text{code_in}[6]$ — positions 2, 3, 6, 7
- $s4 = \text{code_in}[3] \oplus \text{code_in}[4] \oplus \text{code_in}[5] \oplus \text{code_in}[6]$ — positions 4, 5, 6, 7

Error Detection. The reduction-OR operator `|syndrome` evaluates to 1 if any syndrome bit is non-zero. This is a one-line, synthesisable error flag.

Error Correction. The always block first copies `code_in` into `corrected_code`. If the syndrome is non-zero, it inverts the bit at index `syndrome - 1`. The subtraction converts the 1-based position indicated by the syndrome into a 0-based array index. This is a direct hardware implementation of the syndrome correction algorithm.

Data Extraction. The 4 data bits are at positions 3, 5, 6, 7 of the codeword, which map to indices 2, 4, 5, 6 of the `corrected_code` array. The concatenation reconstructs the original `data_in` in ordering: { `d3`, `d2`, `d1`, `d0` }.

4.5 Module: `hamming74_system` (Top-Level)

The top-level module integrates the three leaf modules into a complete data path, from raw input to corrected output, exposing all intermediate signals for observation and debugging.

```
module hamming74_system(
    input  [3:0] data_in,
    input      inject_error,
    input  [2:0] error_pos,

    output [6:0] encoded_data,
    output [6:0] corrupted_data,
    output [6:0] corrected_code,
    output [3:0] decoded_data,
    output [2:0] syndrome,
    output      error_detected
);
```

The data path is a simple linear chain:

1. `data_in` → `hamming74_encoder` → `encoded_wire` (7-bit codeword)
2. `encoded_wire` + `inject_error` + `error_pos` → `error_injector` → `corrupted_wire`
3. `corrupted_wire` → `hamming74_decoder` → `decoded_data`, `syndrome`, `error_detected`

The top level also reconstructs the `corrected_code` output using the syndrome and `corrupted_wire`, mirroring the correction logic inside the decoder. This provides direct observation of the corrected word on a top-level port, which is useful for waveform inspection in simulation:

```
assign corrected_code = (error_detected) ?
    (corrupted_wire ^ (7'b0000001 << (syndrome - 1)))
    : corrupted_wire;
```

When `error_detected` is low (`syndrome = 0`), the `corrected_code` is simply the uncorrupted input. When an error is detected, the correction is applied by XORing a one-hot mask, shifted to the syndrome position.

4.6 Module Connectivity Summary

Signal	Source Module	Sink Module	Description
<code>data_in[3:0]</code>	System Input	Encoder	4 raw data bits
<code>encoded_wire[6:0]</code>	Encoder	Injector	7-bit Hamming codeword
<code>inject_error</code>	System Input	Injector	Enable error injection
<code>error_pos[2:0]</code>	System Input	Injector	Position of injected error
<code>corrupted_wire[6:0]</code>	Injector	Decoder + Top	Possibly corrupted word
<code>syndrome[2:0]</code>	Decoder	Top + Output	Error location indicator
<code>decoded_data[3:0]</code>	Decoder	System Output	Recovered 4-bit data

5. Planned Next Steps

The following enhancements are planned to extend this project into a complete, production-quality digital design repository:

- **Complete Verification Environment.** A SystemVerilog/Verilog testbench (tb/top_tb.v) covering all 16 data inputs across all 7 error positions, including no-error and boundary-condition test cases.
- **Automatic Self-Checking Testbench.** A reference model embedded in the testbench that computes expected outputs and automatically compares them against DUT results, printing a consolidated PASS/FAIL summary at simulation end — no manual waveform inspection required.
- **Single-Click TCL Automation Script.** A TCL script (run_all.tcl) compatible with ModelSim, QuestaSim, and Vivado xsim that compiles all RTL and testbench sources, runs simulation, and reports results in one command.
- **SECDED Extension (Hamming(8,4)).** An eighth overall-parity bit added to achieve Single Error Correcting, Double Error Detecting capability — a standard requirement in memory controller designs.
- **Scalable Hamming(15,11) Variant.** Extension of the same architecture to an 11-bit data, 15-bit codeword configuration using the same syndrome-based correction principle.

6. Conclusion

The Hamming(7,4) system presented in this repository demonstrates how a small number of carefully chosen redundancy bits can provide robust single-bit error correction in hardware. The four-module architecture — encoder, error injector, decoder, and top-level integrator — cleanly separates concerns and maps directly to the theoretical constructs of syndrome decoding.

The mathematical elegance of the Hamming code, where the syndrome equals the binary address of the error position, translates into remarkably compact Verilog: the entire correction is a single indexed bit-flip. This makes the design not only correct but also highly efficient in terms of gate count and timing.

The planned verification environment and TCL automation will transform the project from a standalone RTL exercise into a production-quality, fully self-checking digital design, suitable as a reference implementation or as a foundation for extended work such as Hamming(15,11) or SECDED (Single Error Correcting, Double Error Detecting) variants.